

GUÍA PERSONAL DE ESTUDIO

Mi trabajo DevOps para memorizar

Guion oral de 5 minutos, hoja de repaso y material de consulta

Primero memorizá

Tu aporte, el flujo de GitHub a Azure, tu intervención y el estado actual.

Después consultá

Definiciones, fallas, costos, comandos y decisiones técnicas.

Microsoft Azure - Terraform - GitHub Actions - PostgreSQL - Event Grid

Versión mejorada - Estado verificado el 14/09/2026

1. Las primeras dos diapositivas

No memorices palabra por palabra. Recordá la idea de cada bloque y contala con tus palabras.

Diapositiva 1 - Mi aporte DevOps (0:00 a 1:35)

En pantalla: Azure, Terraform, pipelines y Event Grid.

Para decir: Mi trabajo DevOps fue preparar un ambiente compartido para que Front y Back pudieran integrar y probar la misma versión fuera de sus computadoras. Elegimos Azure por el crédito estudiantil y usamos Chile Central porque la suscripción rechazó Brazil South.

Organicé los recursos del ambiente, dejé la infraestructura escrita en Terraform y configuré automatizaciones en GitHub Actions. Así los cambios pueden revisarse antes de ejecutarse y cada imagen Docker queda identificada con el SHA del commit. También preparé Event Grid para que el backend pueda publicar eventos técnicos.

Mi responsabilidad no es decidir las reglas de negocio. Mi responsabilidad es construir, desplegar, observar y recuperar la plataforma, además de autorizar los cambios que llegan a Azure.

Frase ancla: Preparé el camino seguro entre el código del equipo y un ambiente compartido en Azure.

Diapositiva 2 - Hitos de la bitácora (1:35 a 3:20)

En pantalla: 11/08 plataforma; 13/08 primer despliegue; 25/08 aplicaciones y PostgreSQL; 01/09 Event Grid; 08-14/09 coordinación.

Para decir: El 11 de agosto definimos Azure, Terraform y GitHub Actions. Preparamos el almacenamiento privado del estado de Terraform, ACR, identidades y autenticación federada con OIDC, para no guardar un client secret en GitHub.

El 13 de agosto revisé y autoricé el primer plan y apply. El 25 incorporamos los pipelines de las aplicaciones, sus imágenes Docker y PostgreSQL 16. Resolvimos dos fallas concretas: el wrapper de Maven no tenía permiso de ejecución en Linux y el proxy devolvía 502 por la configuración TLS SNI.

El 1 de septiembre Event Grid aceptó una publicación técnica con HTTP 200. Entre el 8 y el 14 coordinamos la integración: Front fusionó su PR de integración, mientras que el PR de rutas relativas de Front y el PR de API y migraciones de Back seguían abiertos.

Frase ancla: La bitácora demuestra decisiones, problemas, correcciones y resultados verificables.

2. Resultado, alcance y cierre

La última diapositiva muestra qué quedó funcionando y qué todavía no debemos afirmar.

Diapositiva 3 - Resultado y alcance (3:20 a 5:00)

En pantalla: pruebas técnicas, consistencia, control y pendientes.

Para decir: El resultado fue un ambiente development con frontend, backend, PostgreSQL, proxy y Event Grid. Comprobamos respuestas HTTP 200, conexión real del backend con la base y acceso a /api/health tanto de forma directa como por el dominio del frontend.

Después del despliegue ejecuté nuevamente Terraform y el plan terminó en No changes. Eso demuestra que Azure y la configuración declarada quedaron alineados. Antes de cada cambio reviso la versión, las validaciones y el plan; después autorizo el apply, hago smoke tests y registro evidencia.

También controlamos el crédito: Container Apps puede escalar a cero y PostgreSQL se detiene cuando nadie lo usa. Aun así permanecen costos menores de almacenamiento y backups.

Las cuatro tareas DevOps quedaron completas dentro del alcance técnico inicial. Esto no significa que toda la aplicación esté terminada: faltan funcionalidades del equipo, contratos definitivos de eventos y la primera release hacia test.

Frase ancla: Development está operativo; test está preparado, pero la primera release todavía no se ejecutó.

Apertura sugerida

Soy el responsable DevOps del módulo de Obras Públicas. Mi trabajo fue transformar el código del equipo en una plataforma desplegable, repetible y controlada.

Cierre sugerido

Mi aporte no termina cuando el pipeline queda verde: termina cuando la versión responde, la base está conectada y Terraform confirma que no hay diferencias.

Ritmo recomendado

Hablá despacio y hacé una pausa breve después de cada frase ancla. Si olvidás un detalle, volvé a cuatro ideas: ambiente compartido, automatización, autorización y verificación.

3. Lo que realmente tenés que memorizar

Esta página es el resumen para leer antes de presentar.

1. Problema

Cada developer tenía su versión local.
Necesitábamos un ambiente común y reproducible.

2. Solución

Azure ejecuta; Terraform define; GitHub controla;
Docker empaqueta.

3. Tu rol

Vos revisás, autorizás el plan y el apply, probás y guardás evidencia.

4. Development

Recibe los cambios integrados durante el sprint y permite probar al equipo.

5. Test

Recibe una release candidata al cerrar el sprint. El PO valida funcionalmente.

6. Estado

Development operativo. Primera release a test pendiente.

El recorrido que debés poder explicar sin mirar



El código no llega solo a Azure: primero se revisa, se valida y se convierte en una versión identificable; después DevOps autoriza su ejecución.

La frase de oro

Un pipeline verde demuestra que la automatización terminó. Un despliegue completo requiere además probar la aplicación, sus dependencias y la convergencia de Terraform.

4. Responsabilidades, GitHub y GitFlow

Quién hace qué

Frontend y Backend

Front construye la experiencia y consume /api. Back define reglas, endpoints, contratos JSON, autenticación, migraciones y publicación de eventos.

DevOps y Product Owner

DevOps gestiona infraestructura, automatización, secretos, despliegues, logs, costos y rollback. El PO prioriza y valida funcionalmente.

Conceptos de GitHub

Git	Registra cambios y conserva el historial.	GitHub	Comparte repositorios y ejecuta automatizaciones.
Commit	Foto identificada de un conjunto de cambios.	Push	Publica commits en GitHub.
Pull Request	Pedido de incorporación con revisión y checks.	Merge	Integra una rama aprobada en otra.

Función de cada rama

feature/*	Trabajo aislado; nace desde develop y vuelve por PR.
develop	Sprint integrado; se despliega en development.
release/vX.Y.Z	Candidato estable; parte de un commit probado de develop.
main + tag	Versión aprobada y recuperable; no es la rama diaria.

No confundas rama con ambiente

La rama contiene código. El ambiente ejecuta una versión de ese código. develop puede desplegarse en development, pero develop no es un servidor.

5. Azure, Terraform y la bitácora

Resource Group

Agrupar recursos, permisos y costos.

Container Apps

Ejecuta Front y Back en contenedores; puede escalar a cero.

PostgreSQL 16

Conserva datos aun si el backend se reinicia.

ACR

Guarda imágenes Docker identificadas por SHA.

Event Grid

Distribuye eventos a destinos suscriptos.

Log Analytics

Conserva registros para investigar fallas.

Bitácora resumida

Fecha	Hito	Evidencia
11/08	Plataforma	Azure, Terraform, ACR, estado privado, identidades y OIDC.
13/08	Primer despliegue	Plan y apply autorizados; Container Apps y logs creados.
25/08	Aplicaciones y datos	CI, imágenes, PostgreSQL 16 y correcciones de mvnw y proxy.
01/09	Mensajería	Event Grid aceptó una publicación técnica con HTTP 200.
08-14/09	Coordinación	Front PR #3 fusionado; Front PR #2 y Back PR #3 abiertos.

Por qué Terraform importa

El plan permite revisar altas, cambios y bajas antes de ejecutar. El apply aplica exactamente el artefacto revisado. Un plan posterior con No changes demuestra convergencia.

6. Pipelines, imágenes y secretos

Qué ocurre cuando el equipo sube un cambio

1. Prueba local

El developer ejecuta lint, tests, build y migraciones.

2. Pull Request

GitHub repite controles antes del merge.

3. Imagen Docker

La aplicación y sus dependencias se empaquetan juntas.

4. SHA o digest

Identifica exactamente qué versión se despliega.

5. Terraform plan

Calcula los cambios de infraestructura.

6. Autorización

DevOps revisa y habilita el apply.

7. Smoke tests

Se comprueba Front, /api/health, base y proxy.

8. Convergencia

Se repite el plan y debe terminar en No changes.

Secretos e identidad

OIDC

GitHub se autentica en Azure mediante una relación de confianza y sin guardar un client secret permanente.

Secretos

DB_URL, DB_USERNAME y DB_PASSWORD pertenecen al backend y a la infraestructura. Nunca se exponen en Front ni se suben a Git.

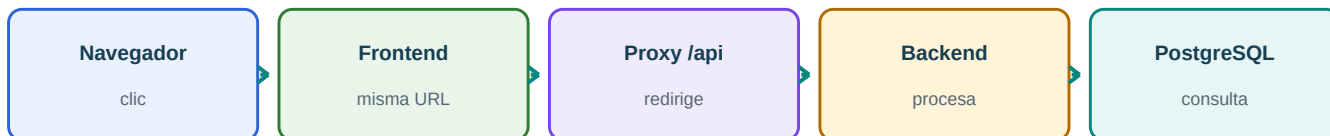
Regla operativa

El mismo commit debe conservarse desde la imagen revisada hasta el despliegue. Si cambia el código, cambia el artefacto y debe volver a validarse.

7. Front, Back, PostgreSQL y eventos

Cómo viaja una llamada /api

Solicitud desde el navegador



Front usa rutas relativas /api. En Azure, Nginx reenvía la solicitud al backend. En local cambia la configuración del proxy, no el código de la aplicación.

Backend debe entregar

Endpoints bajo /api, contratos JSON, códigos HTTP, validaciones, errores, paginación, enums, autenticación y roles.

PostgreSQL y migraciones

La base se configura mediante secretos. Los cambios de estructura deben ser versionados para que todos los ambientes evolucionen igual.

Eventos y Event Grid

Publicación de un evento



El tópico ya acepta publicaciones técnicas. Las suscripciones, filtros y consumidores de negocio no deben configurarse hasta confirmar contratos, nombres, payloads y destinos.

No inventar integraciones

Tener Event Grid creado no significa que existan contratos definitivos. Los componentes pendientes se mantienen explícitamente como pendientes.

8. Despliegue, fallas, costos y test

Procedimiento único de despliegue

- | | |
|--|---|
| <ol style="list-style-type: none"> 1. Identificar rama, commit, imagen y ambiente de destino. 2. Revisar que CI esté aprobado y sin fallos. 3. Autorizar el plan de Terraform. 4. Evaluar altas, cambios y bajas; detenerse ante una eliminación inesperada. | <ol style="list-style-type: none"> 5. Autorizar el apply sobre el mismo commit revisado. 6. Probar frontend HTTP 200, /health, /api/health, proxy y database: up. 7. Comprobar convergencia con un plan posterior No changes. 8. Registrar workflow, SHA, plan, resultado y decisión de rollback. |
|--|---|

Si algo falla

Prueba fallida: corrige el developer. Baja inesperada: se detiene. Apply fallido: revisar logs antes de reintentar. App sin respuesta: health checks y posible rollback.

Cuidar el crédito

Container Apps escala a cero. PostgreSQL se detiene cuando nadie trabaja, aunque continúan almacenamiento y backups. ACR y Event Grid conservan costos menores.

Promoción de development a test

Cierre de un sprint



La versión que llega a test debe ser la misma imagen probada. El PO valida funcionalmente; DevOps observa salud, logs y capacidad de recuperación.

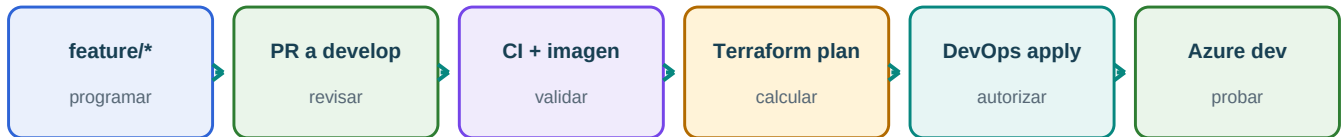
Estado actual

El workflow de test está preparado, pero la primera release todavía no fue ejecutada. Producción no forma parte del alcance actual.

9. Los dos recorridos que resumen tu trabajo

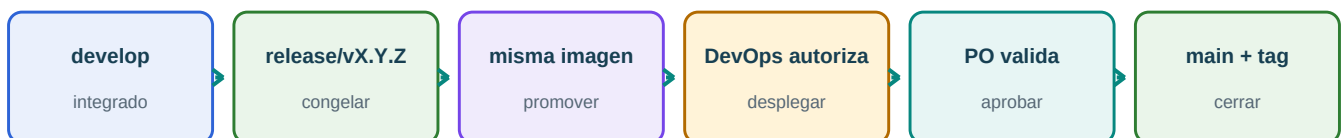
Si podés explicar estas dos líneas, entendés el flujo completo.

Durante el sprint - deployment en development



Los developers trabajan y revisan. GitHub valida y empaqueta. Terraform propone. DevOps decide cuándo ejecutar. El equipo prueba la versión compartida.

Al terminar el sprint - promoción al ambiente test



La release no se reconstruye sin motivo. Se promueve un artefacto conocido, se valida en test y recién entonces queda registrada como versión aprobada.

Hecho

Infraestructura, pipelines, Front, Back, PostgreSQL, proxy, Event Grid técnico, smoke tests y convergencia.

Pendiente

PR de rutas relativas de Front, PR de API y migraciones de Back, contratos de eventos y primera release hacia test.

Evidencia verificada al 14/09/2026

- [Front PR #2](#): abierto hacia develop.
- [Front PR #3](#): fusionado.
- [Back PR #3](#): abierto hacia develop, con CI exitoso.